

AM - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continuous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/"), RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 4132 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```
## [1] "Data without fractional preserved values has 4075 values"
## [1] "57 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite
```

```
# get counts of syllable components in different serial positions
```

```
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))
```

```
syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))
```

```
kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	516	32	124	NA	NA	672
2	61	NA	401	93	106	661
3	291	NA	163	198	14	666
4	287	NA	220	66	34	607
5	220	NA	199	67	32	518
6	190	1	125	72	21	409
7	166	NA	94	28	19	307
8	87	NA	47	21	4	159
9	67	NA	2	NA	7	76

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7678571	0.047619	0.1845238	NA	NA	672
2	0.0922844	NA	0.6066566	0.1406959	0.1603631	661
3	0.4369369	NA	0.2447447	0.2972973	0.0210210	666
4	0.4728171	NA	0.3624382	0.1087315	0.0560132	607
5	0.4247104	NA	0.3841699	0.1293436	0.0617761	518
6	0.4645477	0.002445	0.3056235	0.1760391	0.0513447	409
7	0.5407166	NA	0.3061889	0.0912052	0.0618893	307
8	0.5471698	NA	0.2955975	0.1320755	0.0251572	159
9	0.8815789	NA	0.0263158	NA	0.0921053	76

```
# get percentages only from summary table
```

```
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
```

```
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
```

```
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")
```

```
# pivot to long format for plotting
```

```
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))
```

```
# plot satellite and coda occurrences across position

syll_comp_plot <- ggplot(syll_comp_plot_data,
                        aes(x=pos,y=percent,group=syll_component,
                           color=syll_component,
                           linetype = syll_component,
                           shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype_manual(name="Syllable component", values = palette_linetypes) +
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_pos_of_syll_complexities_in_stimuli.png"), plot=syll_comp_plot)
```

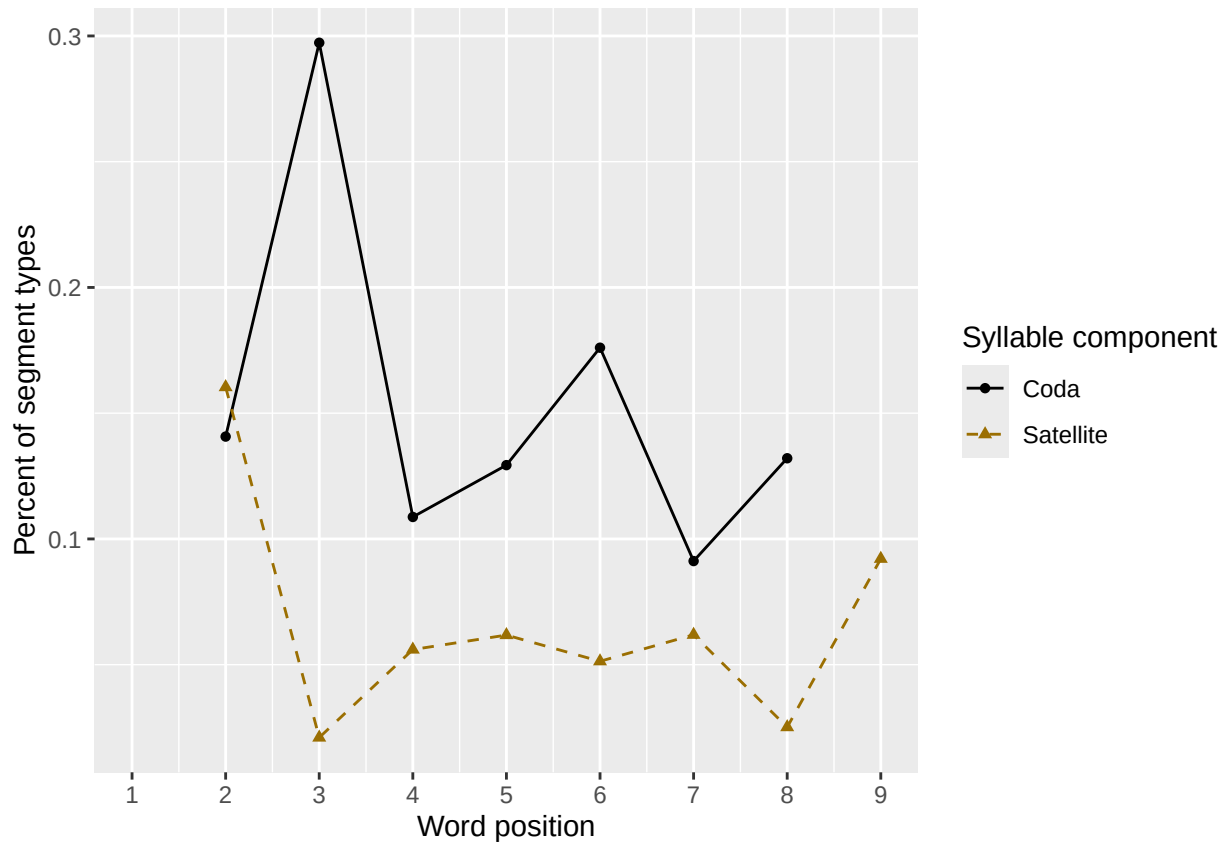
Warning: Removed 3 rows containing missing values or values outside the scale range (`geom\_line()`).

Warning: Removed 3 rows containing missing values or values outside the scale range (`geom\_point()`).

syll_comp_plot

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```



```
# len/pos table
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```

min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.870 0.889 0.833 NA      NA      NA      NA      NA      NA
## 2      5 0.859 0.924 0.857 0.868 NA      NA      NA      NA      NA
## 3      6 0.904 0.910 0.814 0.842 0.772 NA      NA      NA      NA
## 4      7 0.808 0.882 0.837 0.824 0.894 0.865 NA      NA      NA
## 5      8 0.821 0.881 0.867 0.832 0.839 0.860 0.818 NA      NA
## 6      9 0.874 0.871 0.791 0.894 0.805 0.851 0.830 0.779 NA
## 7     10 0.773 0.919 0.853 0.792 0.827 0.747 0.842 0.863 0.75

```

```

# len/pos table
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
"_N_by_len_position.csv"))
pos_len_N_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1      4    54    54    54    NA    NA    NA    NA    NA    NA
## 2      5    92    92    91    91    NA    NA    NA    NA    NA
## 3      6   115   111   113   114   114    NA    NA    NA    NA
## 4      7   104   102   104   102   104   104    NA    NA    NA
## 5      8   145   143   143   143   143   143   143    NA    NA
## 6      9    87    85    86    85    82    87    88    86    NA
## 7     10    75    74    75    72    75    75    76    73    76

```

```

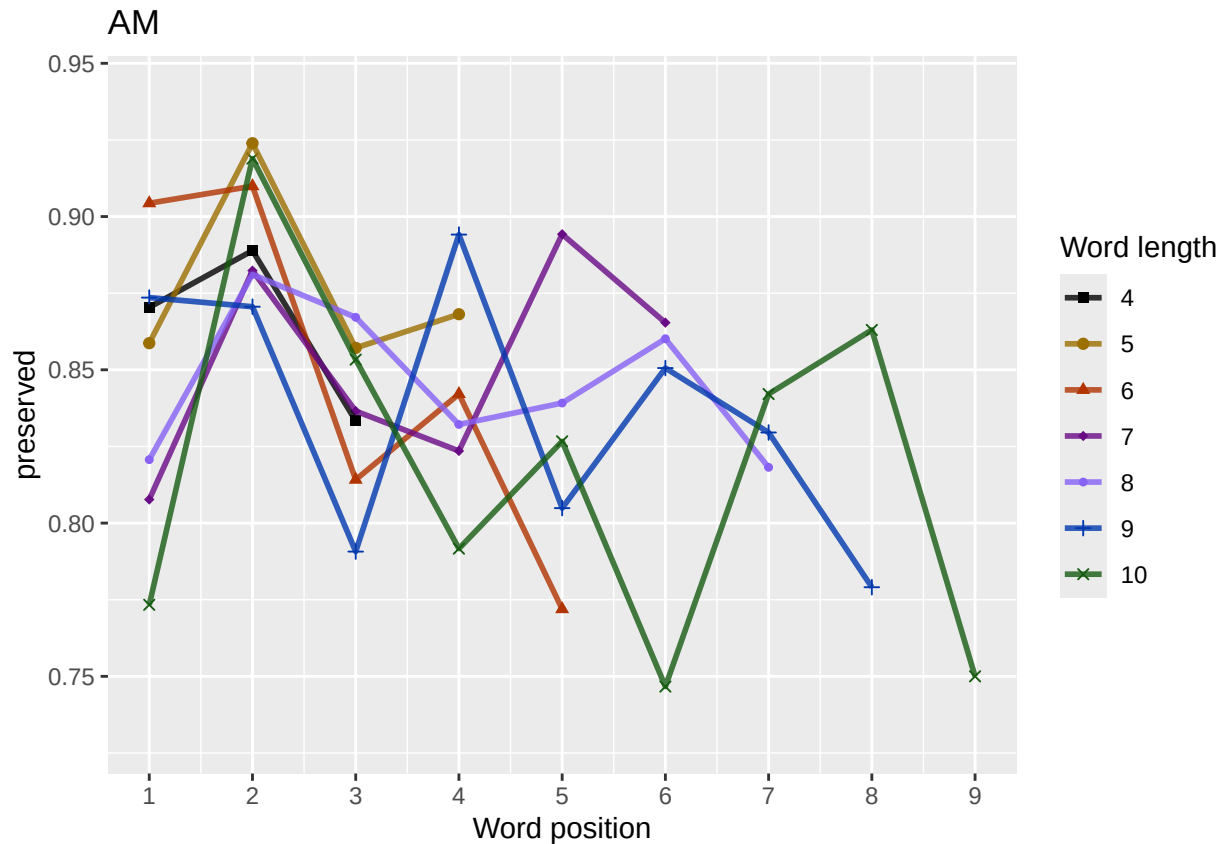
obs_linetypes <- c("solid", "solid", "solid", "solid",
"solid", "solid", "solid", "solid", "solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
pos_len_summary$pos <- factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
paste0(CurPat),
NULL,
c(min_preserved, max_preserved),
palette_values,
shape_values,
obs_linetypes,
pred_linetypes)

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position
```

```
LPMModelEquations<-c("preserved ~ 1",
                      "preserved ~ stimlen",
                      "preserved ~ pos",
                      "preserved ~ stimlen + pos",
                      "preserved ~ stimlen*pos",
                      "preserved ~ I(pos^2)+pos",
                      "preserved ~ stimlen + I(pos^2) + pos",
                      "preserved ~ stimlen * (I(pos^2) + pos)"
)
```

```
LPres<-TestModels(LPMModelEquations,PosDat)
```

```
## *****
```

```
## model index: 4
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```

##
## Coefficients:
## (Intercept)      stimlen      pos
##      2.19054      -0.04218      -0.04452
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4072 Residual
## Null Deviance:      3523
## Residual Deviance: 3512  AIC: 3518
## log likelihood:  -1756.227
## Nagelkerke R2:  0.004473187
## % pres/err predicted correctly:  -1066.911
## % of predictable range [ (model-null)/(1-null) ]:  0.002623937
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      1.9180      -0.0579
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4073 Residual
## Null Deviance:      3523
## Residual Deviance: 3515  AIC: 3519
## log likelihood:  -1757.327
## Nagelkerke R2:  0.003542813
## % pres/err predicted correctly:  -1067.468
## % of predictable range [ (model-null)/(1-null) ]:  0.002104604
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      1.8217507      -0.0066678      0.0007609
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4072 Residual
## Null Deviance:      3523
## Residual Deviance: 3514  AIC: 3520
## log likelihood:  -1757.038
## Nagelkerke R2:  0.003786915
## % pres/err predicted correctly:  -1067.278
## % of predictable range [ (model-null)/(1-null) ]:  0.002281977
## *****
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)      pos

```

```

##      2.108693      -0.039628      -0.004521      -0.005541
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4071 Residual
## Null Deviance:          3523
## Residual Deviance: 3512  AIC: 3520
## log likelihood:  -1756.098
## Nagelkerke R2:  0.004582051
## % pres/err predicted correctly:  -1066.813
## % of predictable range [ (model-null)/(1-null) ]:  0.00271629
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##      2.326878      -0.058737      -0.087776      0.005081
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4071 Residual
## Null Deviance:          3523
## Residual Deviance: 3512  AIC: 3520
## log likelihood:  -1756.159
## Nagelkerke R2:  0.004530625
## % pres/err predicted correctly:  -1066.901
## % of predictable range [ (model-null)/(1-null) ]:  0.002633482
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      2.1937      -0.0652
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4073 Residual
## Null Deviance:          3523
## Residual Deviance: 3517  AIC: 3521
## log likelihood:  -1758.33
## Nagelkerke R2:  0.002693439
## % pres/err predicted correctly:  -1068.055
## % of predictable range [ (model-null)/(1-null) ]:  0.001556162
## *****
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos  stimlen:I(pos^2)      stimlen:I(pos^2)
##      2.797644      -0.131163      0.028269      -0.362846      -0.004329      0.000000
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4069 Residual

```



```
## Null Deviance:      3523
## Residual Deviance: 3511 AIC: 3523
## log likelihood: -1755.501
## Nagelkerke R2: 0.005086895
## % pres/err predicted correctly: -1066.512
## % of predictable range [ (model-null)/(1-null) ]: 0.002997021
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 1.691
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4074 Residual
## Null Deviance:      3523
## Residual Deviance: 3523 AIC: 3525
## log likelihood: -1761.509
## Nagelkerke R2: 1.918291e-16
## % pres/err predicted correctly: -1069.721
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                          AIC=LPRes$AIC,
                          row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                              "_len_pos_model_summary.csv"),row.names = FALSE)

kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2 (Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~	3518.454	0.000000	0.000000	0.00275187	0.00447321	90537	-	-	NA	NA
stimlen + pos							0.0421842	0.0445244		
preserved ~ pos	3518.653	0.199325	0.790514	0.25249083	0.00354189	17988	NA	-	NA	NA
								0.0579044		
preserved ~	3520.076	0.622403	0.34443238	0.222272	0.00378698	21751	NA	0.0007609	NA	NA
I(pos^2) + pos									0.0066678	
preserved ~	3520.196	0.742576	0.8418412	0.11151406	0.00458211	108693	-	-	NA	NA
stimlen + I(pos^2)							0.0396278	0.0055410		0.0045208
+ pos										

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * pos	3520.318	864180	439372	9710834	0.300453	26326878	-	-	0.0050811	NA	NA
							0.058737	0.0877760			
preserved ~ stimlen	3520.662	206139	633185	5080913	2000269	34193662	-	NA	NA	NA	NA
							0.0651992				
preserved ~ stimlen * (I(pos^2) + pos)	3523.002	254859	4510286	9202830	8300508	69797644	-	-	0.0470458	0.0282694	-
							0.131163	23628461		0.0043287	
preserved ~ 1	3525.016	756338	9003756	4501033	73000000	00691468	NA	NA	NA	NA	NA

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ stimlen + pos"
```

```
print(BestLPModel)
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```
##
```

```
## Coefficients:
```

```
## (Intercept)      stimlen          pos
```

```
##      2.19054      -0.04218      -0.04452
```

```
##
```

```
## Degrees of Freedom: 4074 Total (i.e. Null); 4072 Residual
```

```
## Null Deviance: 3523
```

```
## Residual Deviance: 3512 AIC: 3518
```

```
PosDat$LPFitted<-fitted(BestLPModel)
```

```
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],  
NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
```

```
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
```

```
## # Groups:   stimlen [7]
```

```
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`  
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
```

```
## 1      4 0.878 0.874 0.869 NA      NA      NA      NA      NA      NA
```

```
## 2      5 0.874 0.869 0.864 0.858 NA      NA      NA      NA      NA
```

```
## 3      6 0.869 0.864 0.859 0.853 0.847 NA      NA      NA      NA
```

```
## 4      7 0.864 0.859 0.853 0.848 0.842 0.836 NA      NA      NA
```

```
## 5      8 0.859 0.854 0.848 0.842 0.836 0.830 0.824 NA      NA
```

```
## 6      9 0.854 0.848 0.843 0.837 0.830 0.824 0.817 0.811 NA
```

```
## 7     10 0.849 0.843 0.837 0.831 0.824 0.818 0.811 0.804 0.797
```

```

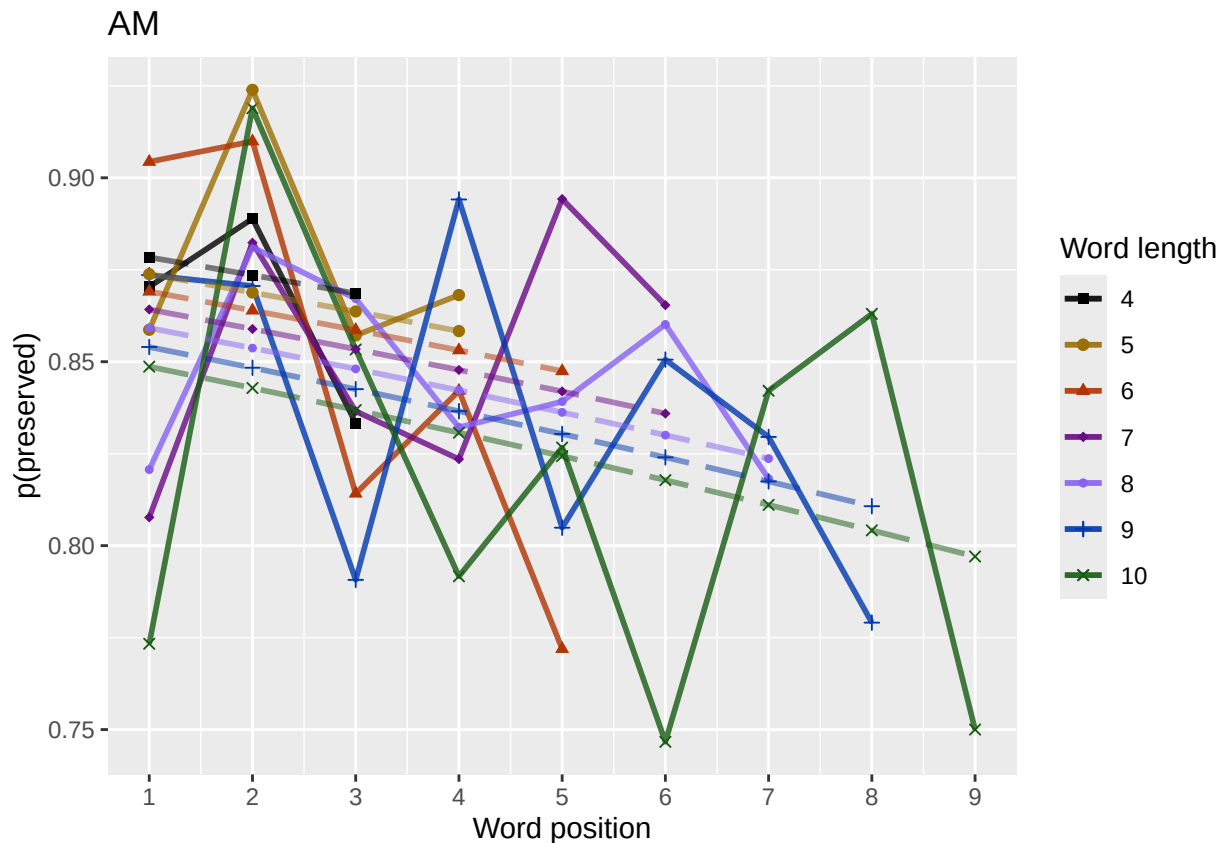
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen, color=stimlen))
# geom_point(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen, color=stimlen)) + ggtitle(paste0("Patient", patient_id))

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
  paste0(PosDat$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"), plot=fitted_len_pos_plot

```



length and position without fragments to see if this changes position² influence

```

# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

```

```

# number responses
resp_num<-0
prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array

# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag

## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1      42   674

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.

print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
  num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 42 / 674 = 6.23 percent"

write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
  CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##    2.16018    -0.05550     0.04259
##
## Degrees of Freedom: 3958 Total (i.e. Null); 3956 Residual
## Null Deviance:      3072
## Residual Deviance: 3067 AIC: 3073
## log likelihood: -1533.642
## Nagelkerke R2: 0.002227409
## % pres/err predicted correctly: -898.3354
## % of predictable range [ (model-null)/(1-null) ]: 0.00123628

```

```

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 1.894
##
## Degrees of Freedom: 3958 Total (i.e. Null); 3958 Residual
## Null Deviance: 3072
## Residual Deviance: 3072 AIC: 3074
## log likelihood: -1536.023
## Nagelkerke R2: 4.113913e-16
## % pres/err predicted correctly: -899.4486
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 2.1655 -0.0355
##
## Degrees of Freedom: 3958 Total (i.e. Null); 3957 Residual
## Null Deviance: 3072
## Residual Deviance: 3070 AIC: 3074
## log likelihood: -1535.225
## Nagelkerke R2: 0.0007468444
## % pres/err predicted correctly: -899.0897
## % of predictable range [ (model-null)/(1-null) ]: 0.0003986203
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos stimlen:pos
## 2.49697 -0.09659 -0.07130 0.01349
##
## Degrees of Freedom: 3958 Total (i.e. Null); 3955 Residual
## Null Deviance: 3072
## Residual Deviance: 3067 AIC: 3075
## log likelihood: -1533.267
## Nagelkerke R2: 0.002577946
## % pres/err predicted correctly: -898.1627
## % of predictable range [ (model-null)/(1-null) ]: 0.001428032
## *****
## model index: 3
##

```

```

## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
## 1.79579      0.02627
##
## Degrees of Freedom: 3958 Total (i.e. Null); 3957 Residual
## Null Deviance: 3072
## Residual Deviance: 3071 AIC: 3075
## log likelihood: -1535.335
## Nagelkerke R2: 0.0006435859
## % pres/err predicted correctly: -899.1358
## % of predictable range [ (model-null)/(1-null) ]: 0.0003474024
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
## 2.1547305   -0.0553346   -0.0003347    0.0453659
##
## Degrees of Freedom: 3958 Total (i.e. Null); 3955 Residual
## Null Deviance: 3072
## Residual Deviance: 3067 AIC: 3075
## log likelihood: -1533.641
## Nagelkerke R2: 0.002227914
## % pres/err predicted correctly: -898.3348
## % of predictable range [ (model-null)/(1-null) ]: 0.001236982
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
## 1.751870   -0.003361    0.054620
##
## Degrees of Freedom: 3958 Total (i.e. Null); 3956 Residual
## Null Deviance: 3072
## Residual Deviance: 3071 AIC: 3077
## log likelihood: -1535.279
## Nagelkerke R2: 0.0006957666
## % pres/err predicted correctly: -899.1074
## % of predictable range [ (model-null)/(1-null) ]: 0.0003789017
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##

```

```
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen:I(pos^2)
##      2.5589209      -0.1165861      -0.0066869      -0.0759895      -0.0001925      0.0001925
##
## Degrees of Freedom: 3958 Total (i.e. Null); 3953 Residual
## Null Deviance:      3072
## Residual Deviance: 3066 AIC: 3078
## log likelihood: -1533.045
## Nagelkerke R2: 0.00278503
## % pres/err predicted correctly: -898.0457
## % of predictable range [ (model-null)/(1-null) ]: 0.001558001
## *****
```

```
NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                                AIC=NoFrag_LPRes$AIC,
                                row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~	3073.283	0.000000	0.000000	0.000000	0.002222	24160181	-	0.0425879	NA	NA
stimlen + pos							0.0554969			
preserved ~ 1	3074.046	0.762465	0.168301	0.017383	0.000000	00893542	NA	NA	NA	NA
preserved ~	3074.449	1.166260	0.055814	0.021420	0.000742	08165509	-	NA	NA	NA
stimlen							0.0355023			
preserved ~	3074.538	1.249988	0.053526	0.013622	0.002572	29496972	-	-	0.013489	NA
stimlen * pos							0.0965863	0.0712998		
preserved ~ pos	3074.670	1.386990	0.049982	0.012720	0.000643	6795795	NA	0.0262745	NA	NA
preserved ~	3075.282	1.998920	0.036807	0.009367	0.002222	29154730	-	0.0453659	NA	NA
stimlen + I(pos^2)							0.0553346			0.0003347
+ pos										
preserved ~	3076.553	3.275448	0.019442	0.004948	0.000695	18751870	NA	0.0546197	NA	NA
I(pos^2) + pos										0.0033607
preserved ~	3078.090	4.806843	0.009040	0.010230	0.000950	20558921	-	-	0.022235	-
stimlen * (I(pos^2)							0.1165861	0.0759895		0.0066869
+ pos)										0.0001925

```
# plot no fragment data
```

```
NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

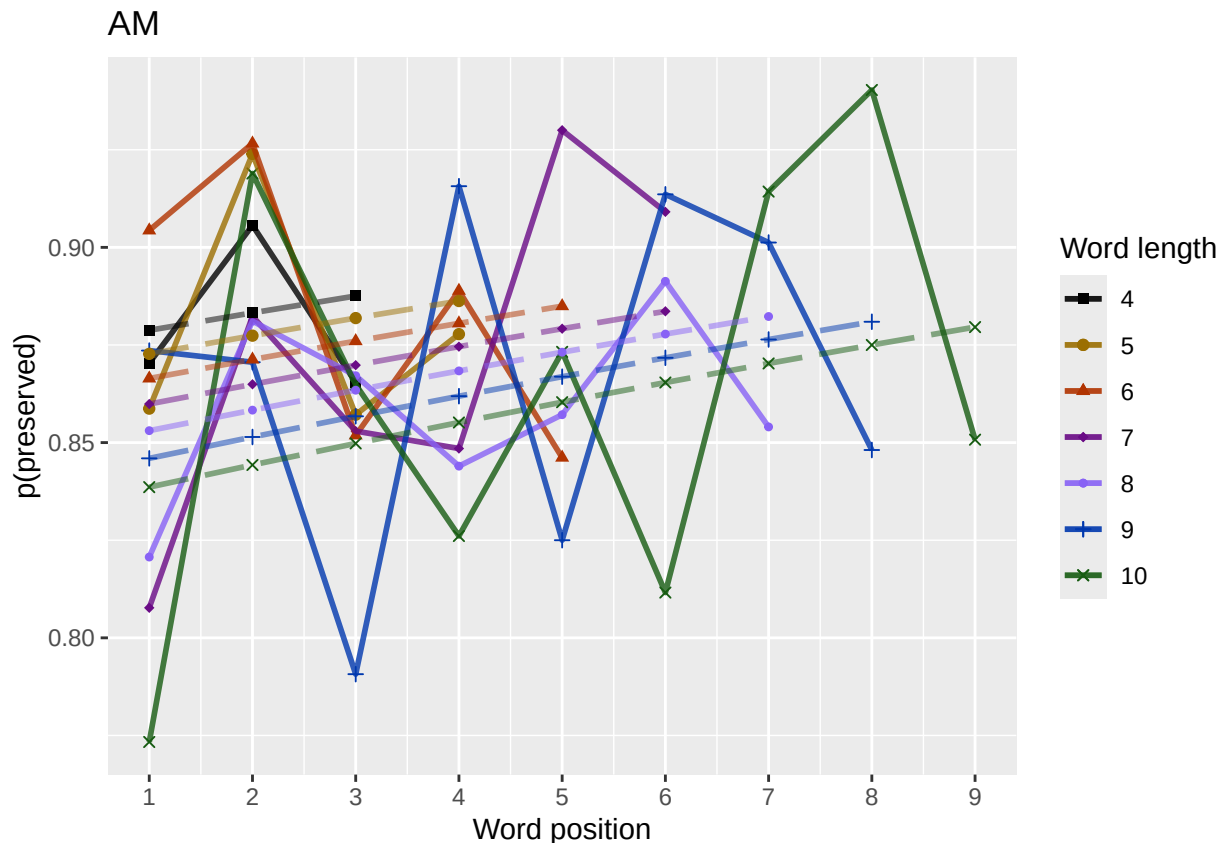
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.879 0.883 0.888 NA      NA      NA      NA      NA      NA
## 2      5 0.873 0.877 0.882 0.886 NA      NA      NA      NA      NA
## 3      6 0.866 0.871 0.876 0.881 0.885 NA      NA      NA      NA
## 4      7 0.860 0.865 0.870 0.875 0.879 0.884 NA      NA      NA
## 5      8 0.853 0.858 0.863 0.868 0.873 0.878 0.882 NA      NA
## 6      9 0.846 0.851 0.857 0.862 0.867 0.872 0.876 0.881 NA
## 7     10 0.839 0.844 0.850 0.855 0.860 0.865 0.870 0.875 0.880
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimlen)) +
#   geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(paste0("Fitted Length vs. Preserved Length by Stimlen"))
```

```
nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
  paste0(NoFragData$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot
```

back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.73 - 0.94"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
```

```
# take away column with length information
```

```
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
```

```
# don't want downward estimates influenced by return upward of U
```

```
# therefore, for downward influence, use only the values before the min
```

```
# take the difference between each value (differences between position proportion correct) **NOTE** pro
```

```
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
```

```
# functions, we need to get differences _first_ and then average those (e.g.
```

```
# if there is an effect of length and we just average position data,
```

```
# later positions) will have a lower average (because of the length effect)
```

```
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] -0.005329532
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] -0.006165014
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
                              2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "No U-shape in this participant"
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentage_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentage_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "no U-shape in this participant"

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small sample sizes)
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)

## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##

```

```

## Coefficients:
## (Intercept)          pos      log_freq pos:log_freq
##      1.91516      -0.05709      0.16124      -0.02134
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4071 Residual
## Null Deviance:      3523
## Residual Deviance: 3500 AIC: 3508
## log likelihood: -1749.809
## Nagelkerke R2:  0.009893103
## % pres/err predicted correctly: -1063.771
## % of predictable range [ (model-null)/(1-null) ]:  0.005557255
## *****
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos      log_freq pos:log_freq
##      1.98694      -0.01106      -0.05369      0.15685      -0.02085
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4070 Residual
## Null Deviance:      3523
## Residual Deviance: 3499 AIC: 3509
## log likelihood: -1749.741
## Nagelkerke R2:  0.00995068
## % pres/err predicted correctly: -1063.737
## % of predictable range [ (model-null)/(1-null) ]:  0.005588288
## *****
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq
##      1.89394      -0.04979      0.07656
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4072 Residual
## Null Deviance:      3523
## Residual Deviance: 3504 AIC: 3510
## log likelihood: -1751.824
## Nagelkerke R2:  0.008192962
## % pres/err predicted correctly: -1064.696
## % of predictable range [ (model-null)/(1-null) ]:  0.004693072
## *****
## model index:  9
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos      log_freq I(pos^2):log_freq
##      1.812e+00      -7.194e-03      6.000e-03      1.636e-01      7.515e-05

```

```

##
## Degrees of Freedom: 4074 Total (i.e. Null); 4069 Residual
## Null Deviance: 3523
## Residual Deviance: 3499 AIC: 3511
## log likelihood: -1749.474
## Nagelkerke R2: 0.01017524
## % pres/err predicted correctly: -1063.567
## % of predictable range [ (model-null)/(1-null) ]: 0.005747621
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos log_freq
## 2.01158 -0.01796 -0.04460 0.07261
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4071 Residual
## Null Deviance: 3523
## Residual Deviance: 3503 AIC: 3511
## log likelihood: -1751.641
## Nagelkerke R2: 0.008347659
## % pres/err predicted correctly: -1064.609
## % of predictable range [ (model-null)/(1-null) ]: 0.004774751
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq log_fr
## 1.978819 -0.011097 0.196854 -0.052710 -0.006047 -0.0
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4069 Residual
## Null Deviance: 3523
## Residual Deviance: 3499 AIC: 3511
## log likelihood: -1749.662
## Nagelkerke R2: 0.01001686
## % pres/err predicted correctly: -1063.713
## % of predictable range [ (model-null)/(1-null) ]: 0.005611586
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq
## 1.98243 -0.01599 0.19748 -0.04454 -0.01595
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4070 Residual
## Null Deviance: 3523

```

```

## Residual Deviance: 3502 AIC: 3512
## log likelihood: -1750.992
## Nagelkerke R2: 0.008895392
## % pres/err predicted correctly: -1064.318
## % of predictable range [ (model-null)/(1-null) ]: 0.005045671
## *****
## model index: 12
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos log_freq I(pos
## 1.8610890 -0.0067483 -0.0068126 0.0047425 0.1612539
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4068 Residual
## Null Deviance: 3523
## Residual Deviance: 3499 AIC: 3513
## log likelihood: -1749.45
## Nagelkerke R2: 0.01019589
## % pres/err predicted correctly: -1063.556
## % of predictable range [ (model-null)/(1-null) ]: 0.005757644
## *****
## model index: 13
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos log_freq
## 1.923009 -0.015148 -0.004873 -0.002593 0.072775
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4070 Residual
## Null Deviance: 3523
## Residual Deviance: 3503 AIC: 3513
## log likelihood: -1751.492
## Nagelkerke R2: 0.008473321
## % pres/err predicted correctly: -1064.499
## % of predictable range [ (model-null)/(1-null) ]: 0.004876637
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq
## 2.01475 -0.04102 0.07256
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4072 Residual
## Null Deviance: 3523
## Residual Deviance: 3507 AIC: 3513
## log likelihood: -1753.746
## Nagelkerke R2: 0.006570685

```

```

## % pres/err predicted correctly: -1065.683
## % of predictable range [ (model-null)/(1-null) ]: 0.003771659
## *****
## model index: 11
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq I(pos^2) pos stimlen:log
## 1.881762 -0.012741 0.201330 -0.005485 0.002722 -0.0
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4069 Residual
## Null Deviance: 3523
## Residual Deviance: 3502 AIC: 3514
## log likelihood: -1750.804
## Nagelkerke R2: 0.009053803
## % pres/err predicted correctly: -1064.184
## % of predictable range [ (model-null)/(1-null) ]: 0.005170962
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq stimlen:log_freq
## 1.98553 -0.03901 0.19790 -0.01601
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4071 Residual
## Null Deviance: 3523
## Residual Deviance: 3506 AIC: 3514
## log likelihood: -1753.091
## Nagelkerke R2: 0.007123456
## % pres/err predicted correctly: -1065.411
## % of predictable range [ (model-null)/(1-null) ]: 0.00402536
## *****
## model index: 10
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq I(pos^2) pos stim
## 1.8544291 -0.0068694 0.2083241 -0.0067050 0.0049298
## log_freq:pos
## -0.0236941
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4067 Residual
## Null Deviance: 3523
## Residual Deviance: 3499 AIC: 3515
## log likelihood: -1749.367
## Nagelkerke R2: 0.01026612
## % pres/err predicted correctly: -1063.53

```



```

## % of predictable range [ (model-null)/(1-null) ]: 0.005781991
## *****
## model index: 17
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      2.19054      -0.04218      -0.04452
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4072 Residual
## Null Deviance:      3523
## Residual Deviance: 3512 AIC: 3518
## log likelihood: -1756.227
## Nagelkerke R2: 0.004473187
## % pres/err predicted correctly: -1066.911
## % of predictable range [ (model-null)/(1-null) ]: 0.002623937
## *****
## model index: 16
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      1.9180      -0.0579
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4073 Residual
## Null Deviance:      3523
## Residual Deviance: 3515 AIC: 3519
## log likelihood: -1757.327
## Nagelkerke R2: 0.003542813
## % pres/err predicted correctly: -1067.468
## % of predictable range [ (model-null)/(1-null) ]: 0.002104604
## *****
## model index: 19
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      1.8217507      -0.0066678      0.0007609
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4072 Residual
## Null Deviance:      3523
## Residual Deviance: 3514 AIC: 3520
## log likelihood: -1757.038
## Nagelkerke R2: 0.003786915
## % pres/err predicted correctly: -1067.278
## % of predictable range [ (model-null)/(1-null) ]: 0.002281977
## *****
## model index: 20

```

```

##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##    2.108693    -0.039628    -0.004521    -0.005541
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4071 Residual
## Null Deviance:      3523
## Residual Deviance: 3512  AIC: 3520
## log likelihood:  -1756.098
## Nagelkerke R2:  0.004582051
## % pres/err predicted correctly:  -1066.813
## % of predictable range [ (model-null)/(1-null) ]:  0.00271629
## *****
## model index:  18
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##    2.326878    -0.058737    -0.087776    0.005081
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4071 Residual
## Null Deviance:      3523
## Residual Deviance: 3512  AIC: 3520
## log likelihood:  -1756.159
## Nagelkerke R2:  0.004530625
## % pres/err predicted correctly:  -1066.901
## % of predictable range [ (model-null)/(1-null) ]:  0.002633482
## *****
## model index:  15
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##    2.1937      -0.0652
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4073 Residual
## Null Deviance:      3523
## Residual Deviance: 3517  AIC: 3521
## log likelihood:  -1758.33
## Nagelkerke R2:  0.002693439
## % pres/err predicted correctly:  -1068.055
## % of predictable range [ (model-null)/(1-null) ]:  0.001556162
## *****
## model index:  21
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)

```

```
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos      stimlen:I(pos^2)      stimlen:I(pos^2)
##      2.797644      -0.131163      0.028269      -0.362846      -0.004329      0.004329
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4069 Residual
## Null Deviance:      3523
## Residual Deviance: 3511 AIC: 3523
## log likelihood: -1755.501
## Nagelkerke R2: 0.005086895
## % pres/err predicted correctly: -1066.512
## % of predictable range [ (model-null)/(1-null) ]: 0.002997021
## *****
## model index: 14
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)
##      1.691
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4074 Residual
## Null Deviance:      3523
## Residual Deviance: 3523 AIC: 3525
## log likelihood: -1761.509
## Nagelkerke R2: 1.918291e-16
## % pres/err predicted correctly: -1069.721
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                          AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                      by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                              RunLabel,"_",CurTask,
                              "_freq_len_pos_model_summary.csv"),row.names = FALSE)

kable(FLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	log_stimlen	log_freq	log_pos	log_freq:I(pos^2)	log_freq:I(pos^2)	log_freq:I(pos^2)	log_freq:I(pos^2)	log_freq:I(pos^2)	log_freq:I(pos^2)
preserved ~ pos *	3507.618	0.0000000	0.370	0.00891516	0.1612425	-	-	NA	NA	NA	NA	NA	NA	NA	NA
log_freq	0.0570892	13385													

Model	AIC Delta	AIC	AICw	NagR ²	Intercept	log_freq	stimlen	log_pos	log_freq	I(pos^2)	pos^2	log_freq	I(pos^2)	len:I(pos^2)
preserved ~ stimlen + pos * log_freq	3509.11	3530.23	3678.05	0.9899	0.9869	0.939	0.15685	-	-	NA	NA	NA	NA	NA
					0.0110562		0.0536902		0.08463					
preserved ~ pos + log_freq	3509.11	3507.35	3620.42	0.9878	0.9838	0.07656	0.07656	-	NA	NA	NA	NA	NA	NA
								0.0497871						
preserved ~ (I(pos^2) + pos) * log_freq	3510.34	3507.21	3590.22	0.9905	0.9916	0.16355	0.00600	0.003	NA	-	0.0000752	NA	NA	NA
								0.0224960	0.0071941					
preserved ~ stimlen + pos + log_freq	3511.28	3506.65	3600.79	0.9929	0.9834	0.07260	0.01	-	NA	NA	NA	NA	NA	NA
					0.0179613		0.0445950							
preserved ~ stimlen * log_freq + pos *	3511.37	3506.25	3603.28	0.9936	0.9900	0.19685	0.01	-	NA	-	NA	NA	NA	NA
					0.0110974	0.0060472	0.0527103	0.0190347						
preserved ~ stimlen * log_freq + pos	3511.48	3505.96	3608.07	0.9887	0.9895	0.19747	0.01	-	NA	NA	NA	NA	NA	NA
					0.0159907	0.0159530	0.0445352							
preserved ~ stimlen + (I(pos^2) + pos) * log_freq	3512.50	3508.02	3610.92	0.9976	0.9910	0.16125	0.00474	NA	-	0.0001056	NA	NA	NA	NA
					0.0067483		0.0224390	0.0068126						
preserved ~ stimlen + I(pos^2) + pos + log_freq	3512.58	3506.35	3684.05	0.9861	0.9819	0.07277	0.01	-	NA	NA	-	NA	NA	NA
					0.0151485		0.0025928		0.0048730					
preserved ~ stimlen + log_freq	3513.51	3507.32	3605.30	0.9904	0.9863	0.07255	0.01	NA	NA	NA	NA	NA	NA	NA
					0.0410188									
preserved ~ stimlen * log_freq + I(pos^2) + pos	3513.56	3509.03	3605.45	0.9928	0.9930	0.20133	0.00272	NA	-	NA	NA	NA	NA	NA
					0.0127411	0.0164225			0.0054847					
preserved ~ stimlen * log_freq	3514.61	3506.35	3676.46	0.9907	0.9912	0.19789	0.01	NA	NA	NA	NA	NA	NA	NA
					0.0390148	0.0160114								
preserved ~ stimlen * log_freq + (I(pos^2) + pos) *	3514.73	3515.09	3628.60	0.9866	0.9910	0.20832	0.00492	-	-	NA	0.0004651	NA	NA	NA
					0.0068694	0.0063431		0.0236940	0.067050					
preserved ~ stimlen + pos	3518.16	3505.87	3643.69	0.9816	0.9812	0.1537	NA	NA	-	NA	NA	NA	NA	NA
					0.0421842		0.0445244							

Model	AIC Delta	AIC	AICw	NagR ²	Intercept	log_stimlen	log_freq	log_pos	log_freq:I(pos)	log_freq:I(pos^2)	log_freq:I(pos^3)	log_freq:I(pos^4)	log_freq:I(pos^5)	len:I(pos^2)
preserved ~ pos	3518.65335	3520.40155	0.87035	0.42879	NA	NA	-	NA	NA	NA	NA	NA	NA	NA
preserved ~ I(pos^2) + pos	3520.17458	3520.70073	0.30378	0.42175	NA	NA	0.00076	0.09	NA	-	NA	NA	NA	NA
preserved ~ stimlen + I(pos^2) + pos	3520.12678	3520.68306	0.87035	0.42879	0.0396278	NA	NA	-	NA	NA	-	NA	NA	NA
preserved ~ stimlen * pos	3520.12800	3520.70073	0.30378	0.42175	0.0587370	NA	NA	-	NA	NA	NA	NA	0.00508	NA
preserved ~ stimlen	3520.66042	3520.72205	0.52393	0.3662	0.0651992	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen * I(pos^2) + pos	3523.16280	3523.43000	0.69528	0.37644	0.1311632	NA	NA	-	NA	NA	0.0282094	NA	0.0470458	0.0043287
preserved ~ 1	3525.17399	3526.06660	0.60060	0.39146	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ pos * log_freq"
```

```
print(BestFLPModel)
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq  pos:log_freq
##      1.91516      -0.05709      0.16124      -0.02134
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4071 Residual
## Null Deviance:      3523
## Residual Deviance: 3500  AIC: 3508
```

```
# do a median split on frequency to plot hf/lf effects (analysis is continuous)
median_freq <- median(PosDat$log_freq)
PosDat$freq_bin[PosDat$log_freq >= median_freq] <- "hf"
PosDat$freq_bin[PosDat$log_freq < median_freq] <- "lf"
```

```
PosDat$FLPFitted <- fitted(BestFLPModel)
```

```
HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]
```

```
HF_Plot <- plot_len_pos_obs_predicted(HFDat, paste0(CurPat, " - ", RunLabel, " - High frequency"), "FLPFitted")
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```

LF_Plot <- plot_len_pos_obs_predicted(LFdat,paste0(CurPat," - ",RunLabel," - Low frequency"),"FLPFitted")

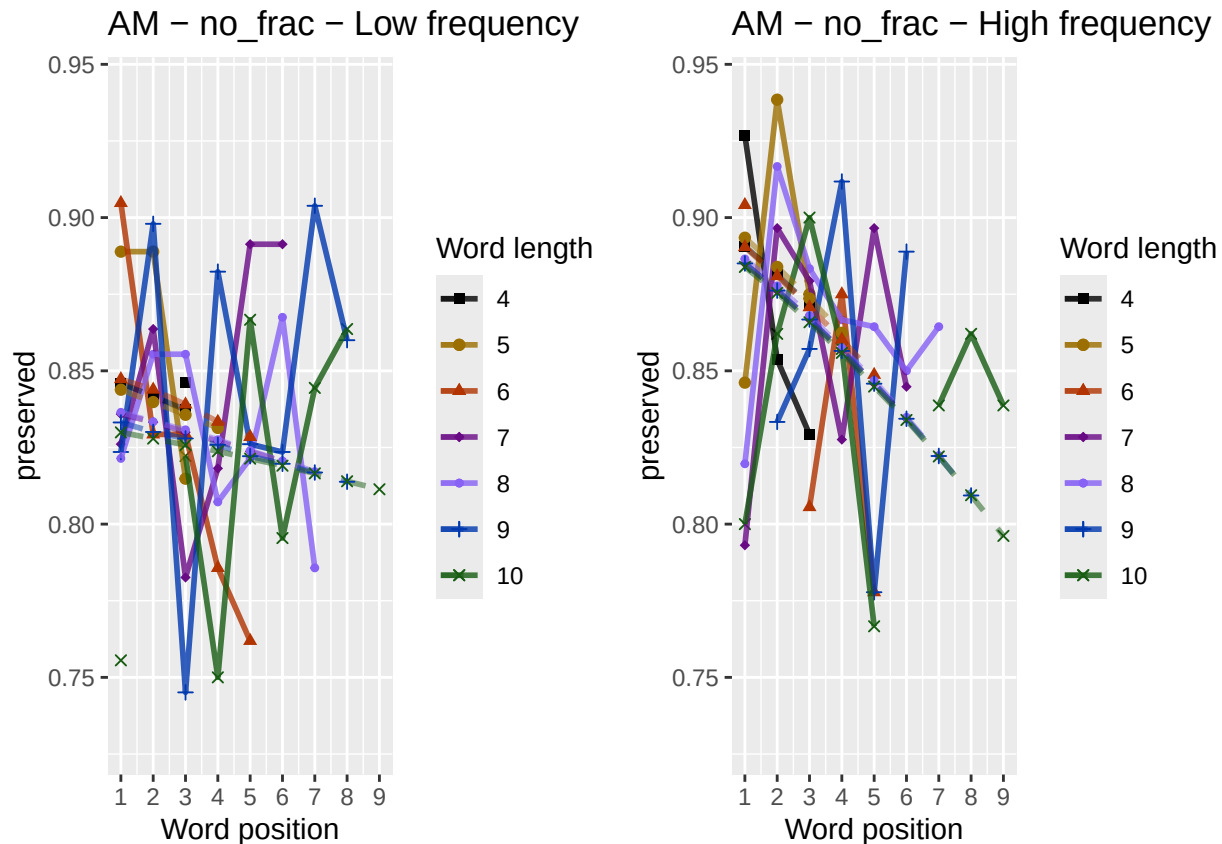
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.

library(ggpubr)
Both_Plots <- ggarrange(LF_Plot,HF_Plot) # labels=c("LF","HF",ncol=2)

## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_line()`).
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_line()`).

ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",
              CurTask,"_frequency_effect_length_pos_wfit.png"),
       device="png",unit="cm",width=30,height=11)
print(Both_Plots)

```



```

# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",

```

```

"preserved ~ 1"
)
MERes<-TestModels(MEModelEquations,PosDat)

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.0539      -0.6622
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4073 Residual
## Null Deviance:      3523
## Residual Deviance: 3321 AIC: 3325
## log likelihood: -1660.591
## Nagelkerke R2: 0.08349509
## % pres/err predicted correctly: -1000.873
## % of predictable range [ (model-null)/(1-null) ]: 0.06430022
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.409      0.128
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4073 Residual
## Null Deviance:      3523
## Residual Deviance: 3493 AIC: 3497
## log likelihood: -1746.349
## Nagelkerke R2: 0.01280807
## % pres/err predicted correctly: -1062.076
## % of predictable range [ (model-null)/(1-null) ]: 0.007139952
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      1.9180      -0.0579
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4073 Residual
## Null Deviance:      3523
## Residual Deviance: 3515 AIC: 3519
## log likelihood: -1757.327
## Nagelkerke R2: 0.003542813
## % pres/err predicted correctly: -1067.468

```

```

## % of predictable range [ (model-null)/(1-null) ]: 0.002104604
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 1.8217507 -0.0066678 0.0007609
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4072 Residual
## Null Deviance: 3523
## Residual Deviance: 3514 AIC: 3520
## log likelihood: -1757.038
## Nagelkerke R2: 0.003786915
## % pres/err predicted correctly: -1067.278
## % of predictable range [ (model-null)/(1-null) ]: 0.002281977
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 2.1937 -0.0652
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4073 Residual
## Null Deviance: 3523
## Residual Deviance: 3517 AIC: 3521
## log likelihood: -1758.33
## Nagelkerke R2: 0.002693439
## % pres/err predicted correctly: -1068.055
## % of predictable range [ (model-null)/(1-null) ]: 0.001556162
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 1.691
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4074 Residual
## Null Deviance: 3523
## Residual Deviance: 3523 AIC: 3525
## log likelihood: -1761.509
## Nagelkerke R2: 1.918291e-16
## % pres/err predicted correctly: -1069.721
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****

```



```

BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
                          AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                      by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                             CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	3325.183	0.0000	1	1	0.083495	2.053936	NA	-	NA	NA	NA
preserved ~ CumPres	3496.698	171.5150	0	0	0.012808	1.409153	0.1279875	NA	NA	NA	NA
preserved ~ pos	3518.653	193.4704	0	0	0.003542	8.917988	NA	NA	NA	-	NA
preserved ~ (I(pos^2) + pos)	3520.076	194.8935	0	0	0.003786	9.821751	NA	NA	-	0.0007609	NA
preserved ~ stimlen	3520.660	195.4772	0	0	0.002693	2.193662	NA	NA	NA	NA	-
preserved ~ 1	3525.017	199.8345	0	0	0.000000	0.691468	NA	NA	NA	NA	NA

```

if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                       family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
                rep(BestMEModelFormulaRnd,RandomSamples))
  AICValues <- c(BestMEModel$aic,RndModelAIC)
}

```

```

BestMEMModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestMEMModelRndDF <- BestMEMModelRndDF %>% arrange(AIC)
BestMEMModelRndDF <- rbind(BestMEMModelRndDF,
                           data.frame(Name=c("Random average"),
                                       AIC=c(mean(RndModelAIC))))
BestMEMModelRndDF <- rbind(BestMEMModelRndDF,
                           data.frame(Name=c("Random SD"),
                                       AIC=c(sd(RndModelAIC))))

write.csv(BestMEMModelRndDF,
          paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                 "_best_main_effects_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary,paste0(TablesDir,CurPat,"_",
                                       RunLabel,"_",CurTask,
                                       "_syllable_component_summary.csv"),row.names = FALSE)
kable(syll_component_summary)

```

syll_component	MeanPres	N
1	0.8605505	545
O	0.8010610	1885
P	0.9090909	33
S	0.6962025	237
V	0.9214545	1375

```

# main effects models for data without satellite positions

keep_components = c("0","V","1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data,MEModelEquations)

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.1187      -0.7494
##
## Degrees of Freedom: 3804 Total (i.e. Null); 3803 Residual

```

```

## Null Deviance:      3176
## Residual Deviance: 2981 AIC: 2985
## log likelihood: -1490.278
## Nagelkerke R2: 0.08836218
## % pres/err predicted correctly: -886.2
## % of predictable range [ (model-null)/(1-null) ]: 0.06977961
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.4754      0.1365
##
## Degrees of Freedom: 3804 Total (i.e. Null); 3803 Residual
## Null Deviance:      3176
## Residual Deviance: 3147 AIC: 3151
## log likelihood: -1573.674
## Nagelkerke R2: 0.01314758
## % pres/err predicted correctly: -945.8794
## % of predictable range [ (model-null)/(1-null) ]: 0.007206305
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      1.77536      -0.01466      0.07126
##
## Degrees of Freedom: 3804 Total (i.e. Null); 3802 Residual
## Null Deviance:      3176
## Residual Deviance: 3166 AIC: 3172
## log likelihood: -1582.879
## Nagelkerke R2: 0.004641461
## % pres/err predicted correctly: -950.0614
## % of predictable range [ (model-null)/(1-null) ]: 0.00282148
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      1.98714      -0.05793
##
## Degrees of Freedom: 3804 Total (i.e. Null); 3803 Residual
## Null Deviance:      3176
## Residual Deviance: 3168 AIC: 3172
## log likelihood: -1584.155

```

```
## Nagelkerke R2: 0.00345941
## % pres/err predicted correctly: -950.807
## % of predictable range [ (model-null)/(1-null) ]: 0.002039737
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      2.17152      -0.05361
##
## Degrees of Freedom: 3804 Total (i.e. Null); 3803 Residual
## Null Deviance: 3176
## Residual Deviance: 3172 AIC: 3176
## log likelihood: -1585.959
## Nagelkerke R2: 0.001786735
## % pres/err predicted correctly: -951.7825
## % of predictable range [ (model-null)/(1-null) ]: 0.001016996
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.759
##
## Degrees of Freedom: 3804 Total (i.e. Null); 3804 Residual
## Null Deviance: 3176
## Residual Deviance: 3176 AIC: 3178
## log likelihood: -1587.883
## Nagelkerke R2: 5.884939e-16
## % pres/err predicted correctly: -952.7524
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
write.csv(SimpSyllMEAICSummary,
  paste0(TablesDir, CurPat, "_", RunLabel, "_",
    CurTask,
    "_OV1_main_effects_model_summary.csv"),
  row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	2984.550	0.0000	1	1	0.0883622	22.118717	NA	-	NA	NA	NA
preserved ~ CumPres	3151.348	66.7915	0	0	0.0131476	6.475438	0.1365059	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos)	3171.758	87.2021	0	0	0.0046415	15.775356	NA	NA	-	0.0712570	NA
									0.0146572		

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ pos	3172.310	87.7535	0	0	0.003459	1.987139	NA	NA	NA	-	NA
										0.0579259	
preserved ~ stimlen	3175.917	91.3609	0	0	0.001786	2.171521	NA	NA	NA	NA	-
											0.0536119
preserved ~ 1	3177.767	93.2106	0	0	0.000000	0.759029	NA	NA	NA	NA	NA

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0","V")
OVDData <- PosDat[PosDat$syll_component %in% keep_components,]
OVDData <- OVDData %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OVDData$CumPres <- CalcCumPres(OVDData)
OVDData$CumErr <- CalcCumErrFromPreserved(OVDData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVDData,MEModelEquations)
```

```
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.0643      -0.7851
##
## Degrees of Freedom: 3259 Total (i.e. Null); 3258 Residual
## Null Deviance:      2735
## Residual Deviance: 2595 AIC: 2599
## log likelihood: -1297.483
## Nagelkerke R2: 0.07412526
## % pres/err predicted correctly: -773.3461
## % of predictable range [ (model-null)/(1-null) ]: 0.05897811
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.5444      0.1161
##
## Degrees of Freedom: 3259 Total (i.e. Null); 3258 Residual
## Null Deviance:      2735
## Residual Deviance: 2722 AIC: 2726
## log likelihood: -1360.757
## Nagelkerke R2: 0.007356323
```

```

## % pres/err predicted correctly: -818.487
## % of predictable range [ (model-null)/(1-null) ]: 0.004120836
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 1.78295 -0.01590 0.07443
##
## Degrees of Freedom: 3259 Total (i.e. Null); 3257 Residual
## Null Deviance: 2735
## Residual Deviance: 2724 AIC: 2730
## log likelihood: -1361.801
## Nagelkerke R2: 0.006233228
## % pres/err predicted correctly: -818.7265
## % of predictable range [ (model-null)/(1-null) ]: 0.003829695
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 2.00641 -0.06544
##
## Degrees of Freedom: 3259 Total (i.e. Null); 3258 Residual
## Null Deviance: 2735
## Residual Deviance: 2726 AIC: 2730
## log likelihood: -1363.203
## Nagelkerke R2: 0.004723009
## % pres/err predicted correctly: -819.5682
## % of predictable range [ (model-null)/(1-null) ]: 0.002806906
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 2.22458 -0.06187
##
## Degrees of Freedom: 3259 Total (i.e. Null); 3258 Residual
## Null Deviance: 2735
## Residual Deviance: 2731 AIC: 2735
## log likelihood: -1365.344
## Nagelkerke R2: 0.002415107
## % pres/err predicted correctly: -820.7369
## % of predictable range [ (model-null)/(1-null) ]: 0.001386673
## *****

```

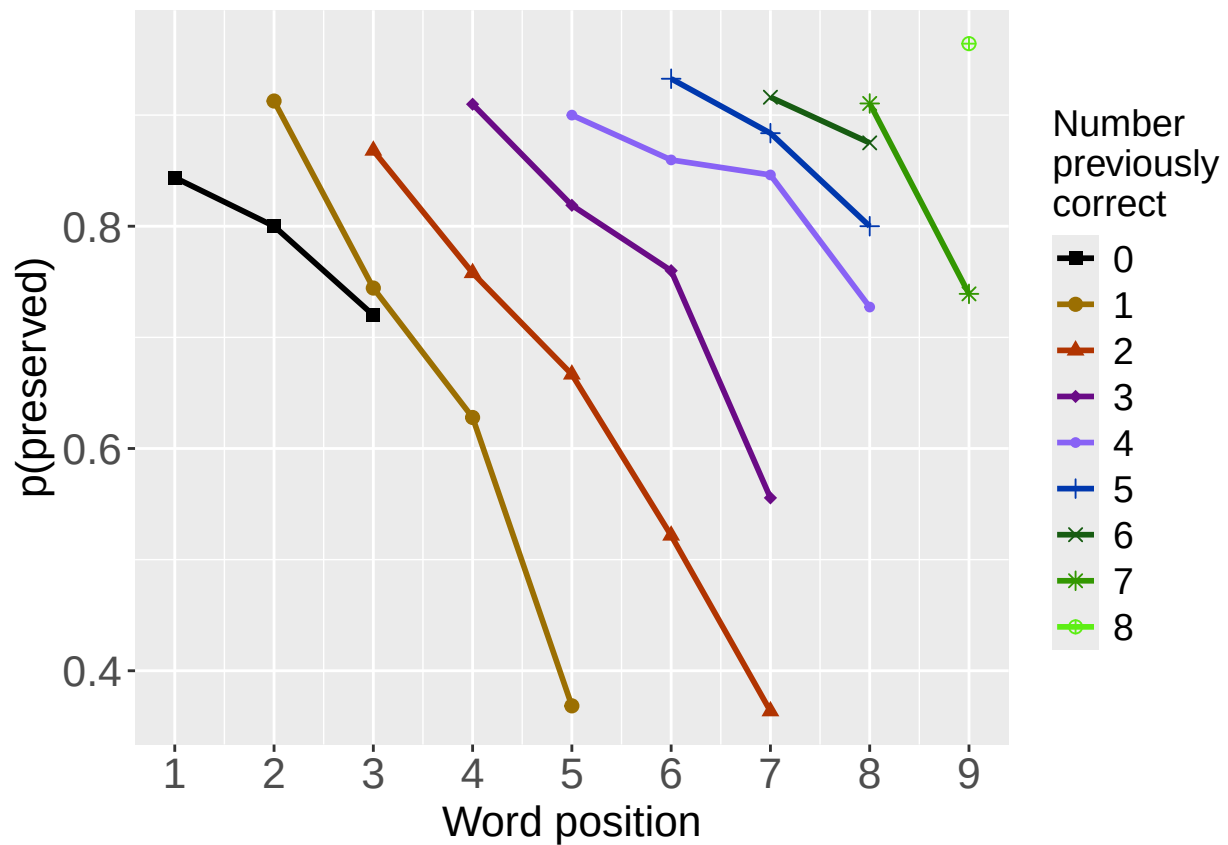
```
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 1.749
##
## Degrees of Freedom: 3259 Total (i.e. Null); 3259 Residual
## Null Deviance: 2735
## Residual Deviance: 2735 AIC: 2737
## log likelihood: -1367.581
## Nagelkerke R2: -3.910197e-16
## % pres/err predicted correctly: -821.8779
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                 "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	2598.967	0.0000	1	1	0.0741253	3.064253	NA	-	NA	NA	NA
preserved ~ CumPres	2725.514	26.5475	0	0	0.0073563	3.544360	0.1161239	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos)	2729.602	30.6347	0	0	0.0062332	3.782947	NA	NA	-	0.0744325	NA
preserved ~ pos	2730.406	31.4392	0	0	0.0047230	3.006408	NA	NA	NA	-	NA
preserved ~ stimlen	2734.687	35.7203	0	0	0.0024152	2.224578	NA	NA	NA	NA	-
preserved ~ 1	2737.161	38.1942	0	0	0.0000000	0.749110	NA	NA	NA	NA	NA

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

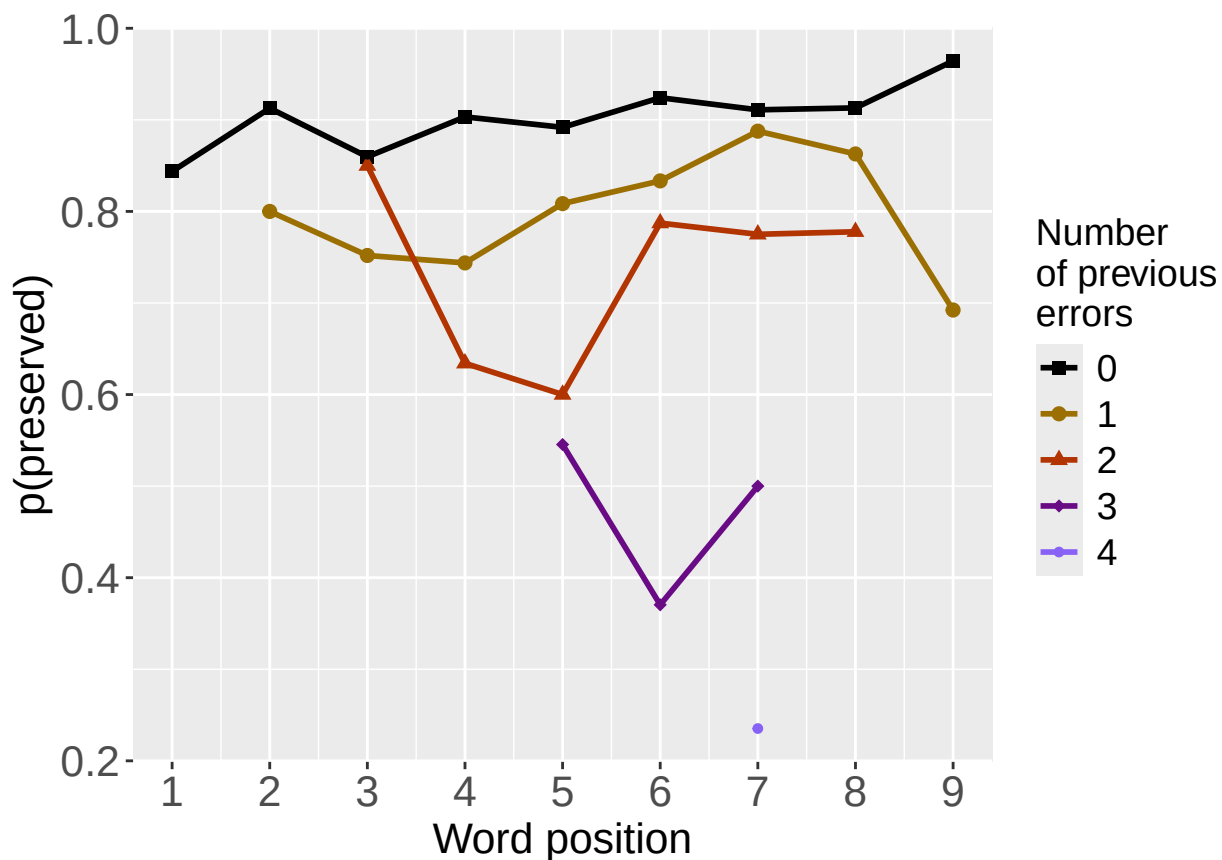
```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```

```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

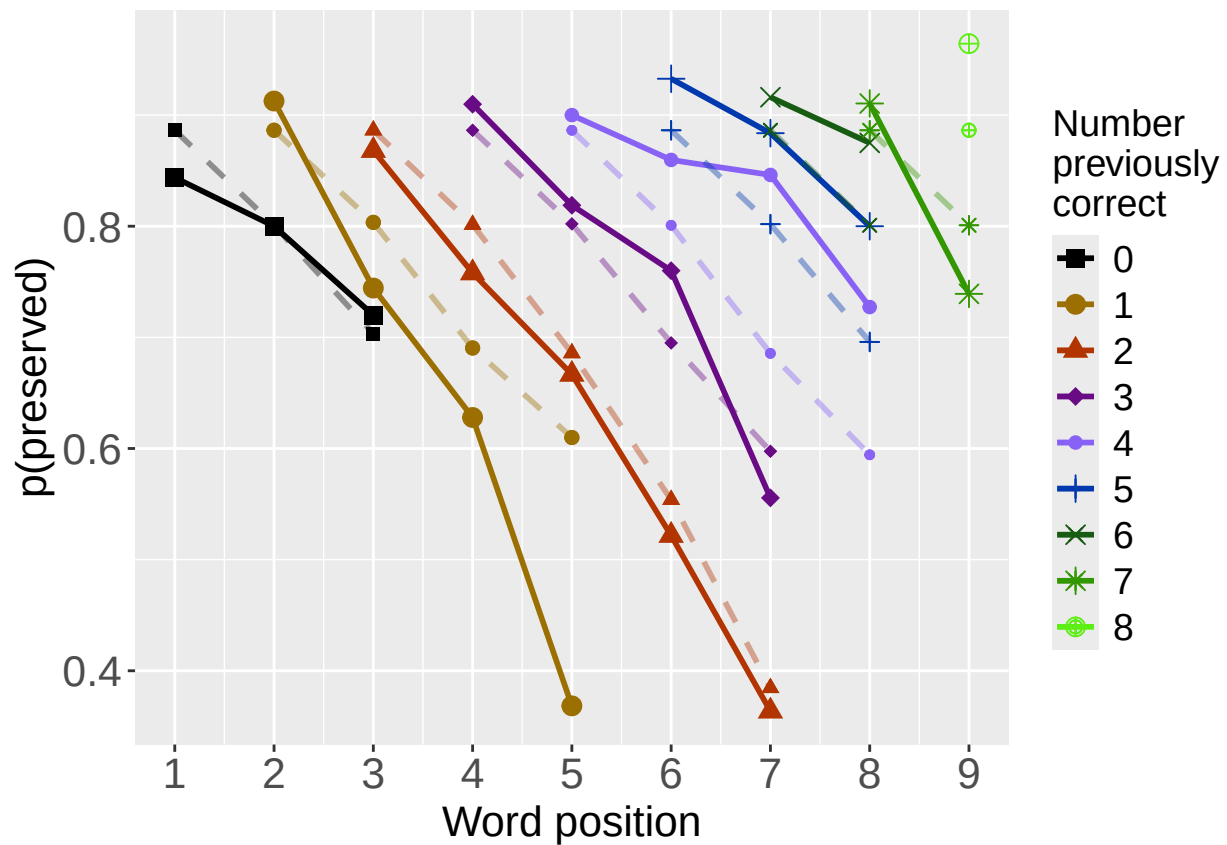
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

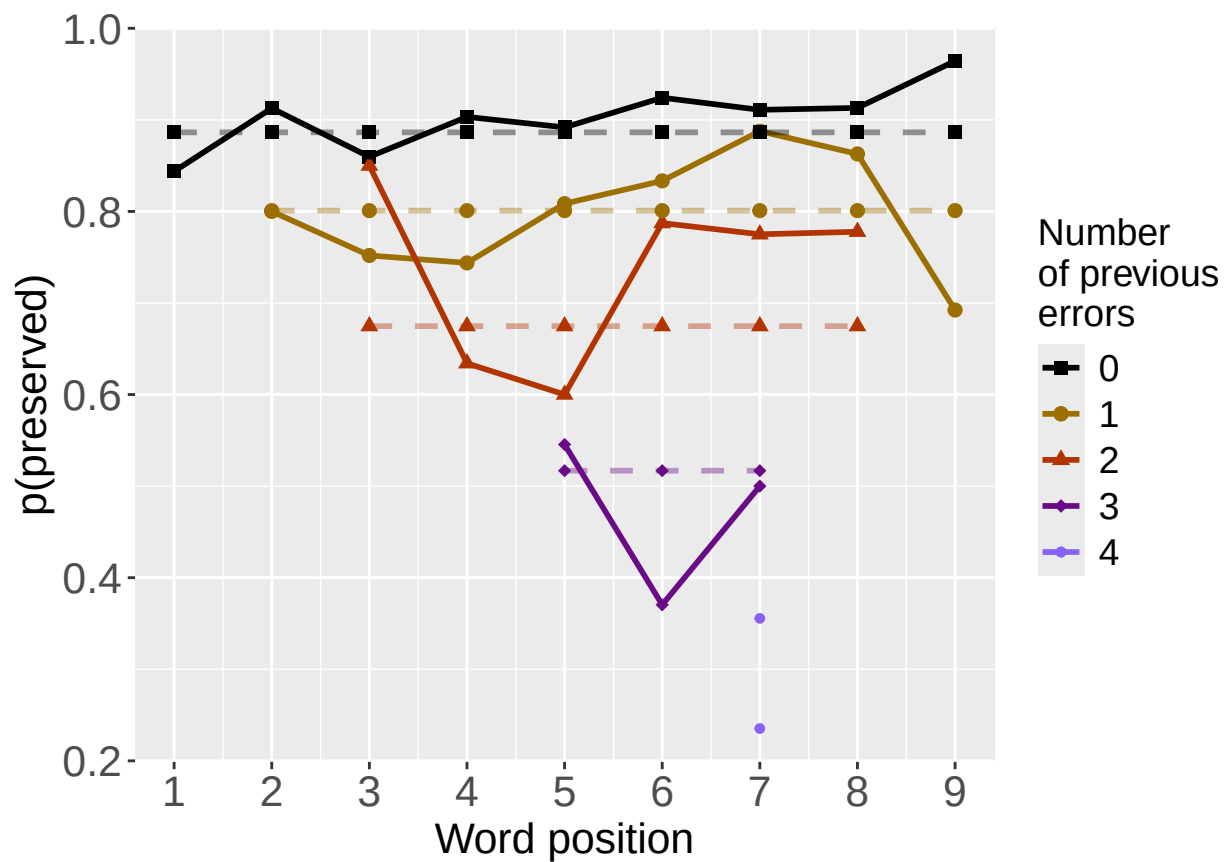
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

```

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)         pos
##      1.7096      -0.7785      -0.0011      0.1112
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4071 Residual
## Null Deviance:      3523
## Residual Deviance: 3304  AIC: 3312
## log likelihood:  -1651.812
## Nagelkerke R2:  0.09056502
## % pres/err predicted correctly:  -996.0763
## % of predictable range [ (model-null)/(1-null) ]:  0.06878042
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.0539      -0.6622
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4073 Residual
## Null Deviance:      3523
## Residual Deviance: 3321  AIC: 3325
## log likelihood:  -1660.591
## Nagelkerke R2:  0.08349509
## % pres/err predicted correctly:  -1000.873
## % of predictable range [ (model-null)/(1-null) ]:  0.06430022
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      1.8217507      -0.0066678      0.0007609
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4072 Residual
## Null Deviance:      3523
## Residual Deviance: 3514  AIC: 3520
## log likelihood:  -1757.038
## Nagelkerke R2:  0.003786915
## % pres/err predicted correctly:  -1067.278
## % of predictable range [ (model-null)/(1-null) ]:  0.002281977
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	3311.624	0.00000	1.0000000	0.9988641	0.0905650	1.709601	-0.778535	-0.0011005	0.1112184
preserved ~ CumErr	3325.183	13.55837	0.0011372	0.0011359	0.0834951	2.053936	-0.662168	NA	NA
preserved ~ I(pos^2) + pos	3520.076	208.45184	0.0000000	0.0000000	0.0037869	1.821751	NA	-0.0066678	0.0007609

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 2.0539 -0.6622
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4073 Residual
## Null Deviance: 3523
## Residual Deviance: 3321 AIC: 3325
## log likelihood: -1660.591
## Nagelkerke R2: 0.08349509
## % pres/err predicted correctly: -1000.873
## % of predictable range [ (model-null)/(1-null) ]: 0.06430022
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr stimlen
## 1.93077 -0.66901 0.01653
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4072 Residual
## Null Deviance: 3523
## Residual Deviance: 3321 AIC: 3327
## log likelihood: -1660.411
## Nagelkerke R2: 0.08364046
## % pres/err predicted correctly: -1000.688
## % of predictable range [ (model-null)/(1-null) ]: 0.06447354
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      2.1937      -0.0652
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4073 Residual
## Null Deviance:      3523
## Residual Deviance: 3517  AIC: 3521
## log likelihood:  -1758.33
## Nagelkerke R2:  0.002693439
## % pres/err predicted correctly:  -1068.055
## % of predictable range [ (model-null)/(1-null) ]:  0.001556162
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr	3325.183	0.000000	1.0000000	0.694207	0.0834951	2.053936	- 0.6621680	NA
preserved ~ CumErr + stimlen	3326.823	1.639723	0.4404926	0.305793	0.0836405	1.930768	- 0.6690146	0.0165282
preserved ~ stimlen	3520.660	195.477203	0.0000000	0.000000	0.0026934	2.193662	NA	- 0.0651992

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      1.7407      -0.6738      0.1440
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4072 Residual
## Null Deviance:      3523
## Residual Deviance: 3287  AIC: 3293
## log likelihood:  -1643.434
## Nagelkerke R2:  0.09728387
## % pres/err predicted correctly:  -990.6031
## % of predictable range [ (model-null)/(1-null) ]:  0.07389215
## *****
## model index:  1
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.0539      -0.6622
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4073 Residual
## Null Deviance:      3523
## Residual Deviance: 3321  AIC: 3325
## log likelihood:  -1660.591
## Nagelkerke R2:  0.08349509
## % pres/err predicted correctly:  -1000.873
## % of predictable range [ (model-null)/(1-null) ]:  0.06430022
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.409      0.128
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4073 Residual
## Null Deviance:      3523
## Residual Deviance: 3493  AIC: 3497
## log likelihood:  -1746.349
## Nagelkerke R2:  0.01280807
## % pres/err predicted correctly:  -1062.076
## % of predictable range [ (model-null)/(1-null) ]:  0.007139952
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr + CumPres	3292.868	0.00000	1e+00	0.9999999	0.0972839	1.740687	- 0.6738377	0.1439872
preserved ~ CumErr	3325.183	32.31525	1e-07	0.0000001	0.0834951	2.053936	- 0.6621680	NA
preserved ~ CumPres	3496.698	203.83022	0e+00	0.0000000	0.0128081	1.409153	NA	0.1279875

```
#####
# level 2 -- Add linear position (NOT quadratic)
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```



```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos
##      1.7246      -0.7787      0.1018
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4072 Residual
## Null Deviance:      3523
## Residual Deviance: 3304 AIC: 3310
## log likelihood: -1651.819
## Nagelkerke R2: 0.09055953
## % pres/err predicted correctly: -996.0732
## % of predictable range [ (model-null)/(1-null) ]: 0.06878338
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.0539      -0.6622
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4073 Residual
## Null Deviance:      3523
## Residual Deviance: 3321 AIC: 3325
## log likelihood: -1660.591
## Nagelkerke R2: 0.08349509
## % pres/err predicted correctly: -1000.873
## % of predictable range [ (model-null)/(1-null) ]: 0.06430022
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      1.9180      -0.0579
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4073 Residual
## Null Deviance:      3523
## Residual Deviance: 3515 AIC: 3519
## log likelihood: -1757.327
## Nagelkerke R2: 0.003542813
## % pres/err predicted correctly: -1067.468
## % of predictable range [ (model-null)/(1-null) ]: 0.002104604
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr	3309.638	0.00000	1.0000000	0.999579	0.0905595	1.724633	-	0.1017997
+ pos							0.7786693	
preserved ~ CumErr	3325.183	15.54471	0.0004212	0.000421	0.0834951	2.053936	-	NA
							0.6621680	
preserved ~ pos	3518.653	209.01510	0.0000000	0.000000	0.0035428	1.917988	NA	-
								0.0579044

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary, paste0(TablesDir, CurPat, "_",
                                RunLabel, "_", CurTask,
                                "_main_effects_plus_one_model_summary.csv"), row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + CumPres	3292.868	0.0000001	0.0000000	0.9999999	0.0972839	1.740687	-	NA	NA	NA	0.1439872
							0.6738377				
preserved ~ CumErr + pos	3309.638	0.0000001	0.0000000	0.9995790	0.0905595	1.724633	-	NA	0.1017997	NA	NA
							0.7786693				
preserved ~ CumErr + I(pos^2) + pos	3311.629	0.0000001	0.0000000	0.9988640	0.0905650	1.709601	-	-	0.1112184	NA	NA
							0.7785350	0.0011005			
preserved ~ CumErr	3325.183	15.54471	0.0011372	0.0011330	0.0834951	2.053936	-	NA	NA	NA	NA
							0.6621680				
preserved ~ CumErr	3325.180	0.0000001	0.0000000	0.6942070	0.0834951	2.053936	-	NA	NA	NA	NA
							0.6621680				
preserved ~ CumErr	3325.183	2.315253	0.0000001	0.0000000	0.0834951	2.053936	-	NA	NA	NA	NA
							0.6621680				
preserved ~ CumErr	3325.183	15.54471	0.0004212	0.0004210	0.0834951	2.053936	-	NA	NA	NA	NA
							0.6621680				
preserved ~ CumErr + stimlen	3326.823	6.6397230	0.4404926	0.3057930	0.0836405	1.930768	-	NA	NA	0.0165282	NA
							0.6690146				
preserved ~ CumPres	3496.692	1203.830203	0.0000000	0.0000000	0.0012808	1.1409153	NA	NA	NA	NA	0.1279875
preserved ~ pos	3518.653	209.01510	0.0000000	0.0000000	0.0035428	1.8917988	NA	NA	-	NA	NA
									0.0579044		
preserved ~ I(pos^2) + pos	3520.070	1008.451841	0.0000000	0.0000000	0.0037869	1.821751	NA	-	0.0007609	NA	NA
							0.0066678				
preserved ~ stimlen	3520.660	95.477203	0.0000000	0.0000000	0.0026934	1.193662	NA	NA	NA	-	NA
										0.0651992	

explore influence of frequency and length

```
if(grepl("stimlen", BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + log_freq")
  )
}
```

```

}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq"),
    paste0(BestModelFormulaL2," + stimlen"),
    paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)

```

```

## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      log_freq
##      1.73043      -0.65924      0.14682      0.04764
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4071 Residual
## Null Deviance:      3523
## Residual Deviance: 3283 AIC: 3291
## log likelihood: -1641.465
## Nagelkerke R2: 0.09885905
## % pres/err predicted correctly: -989.6327
## % of predictable range [ (model-null)/(1-null) ]: 0.07479847
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen      log_freq
##      1.91929      -0.65077      0.15382     -0.02731      0.04143
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4070 Residual
## Null Deviance:      3523
## Residual Deviance: 3282 AIC: 3292
## log likelihood: -1641.05
## Nagelkerke R2: 0.09919034
## % pres/err predicted correctly: -989.4226
## % of predictable range [ (model-null)/(1-null) ]: 0.07499468
## *****
## model index: 4

```

```

##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen
##      2.02086      -0.65828      0.15510      -0.04084
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4071 Residual
## Null Deviance:      3523
## Residual Deviance: 3285  AIC: 3293
## log likelihood:  -1642.428
## Nagelkerke R2:  0.09808821
## % pres/err predicted correctly:  -990.0913
## % of predictable range [ (model-null)/(1-null) ]:  0.0743702
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      1.7407      -0.6738      0.1440
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4072 Residual
## Null Deviance:      3523
## Residual Deviance: 3287  AIC: 3293
## log likelihood:  -1643.434
## Nagelkerke R2:  0.09728387
## % pres/err predicted correctly:  -990.6031
## % of predictable range [ (model-null)/(1-null) ]:  0.07389215
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.691
##
## Degrees of Freedom: 4074 Total (i.e. Null);  4074 Residual
## Null Deviance:      3523
## Residual Deviance: 3523  AIC: 3525
## log likelihood:  -1761.509
## Nagelkerke R2:  1.918291e-16
## % pres/err predicted correctly:  -1069.721
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****

```

```

BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

```

```

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres	log_freq	stimlen
preserved ~ CumErr + CumPres + log_freq	3290.929	0.0000001	0.0000000	0.4314911	0.0988590	0.730434	-	0.1468230	0.0476437	NA
preserved ~ CumErr + CumPres + stimlen + log_freq	3292.100	1.1711800	0.5567770	0.2402447	0.0991903	0.919293	-	0.1538150	0.0414281	-
preserved ~ CumErr + CumPres + stimlen	3292.857	1.9278310	0.3813906	0.1645690	0.0980882	0.20857	-	0.1550972	NA	-
preserved ~ CumErr + CumPres	3292.868	1.9384910	0.3793692	0.1636945	0.0972839	0.740687	-	0.1439872	NA	NA
preserved ~ 1	3525.012	34.0881970	0.0000000	0.0000000	0.0000000	0.691468	NA	NA	NA	NA

```

# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

```

```

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions

```

```

## Single term deletions
##
## Model:
## preserved ~ CumErr + CumPres + log_freq
##      Df Deviance   AIC
## CumErr    1   3476.3 3482.3
## CumPres    1   3318.6 3324.6
## log_freq    1   3286.9 3292.9
## <none>      3282.9 3290.9

```

```

#####
# Single deletions from best model

```

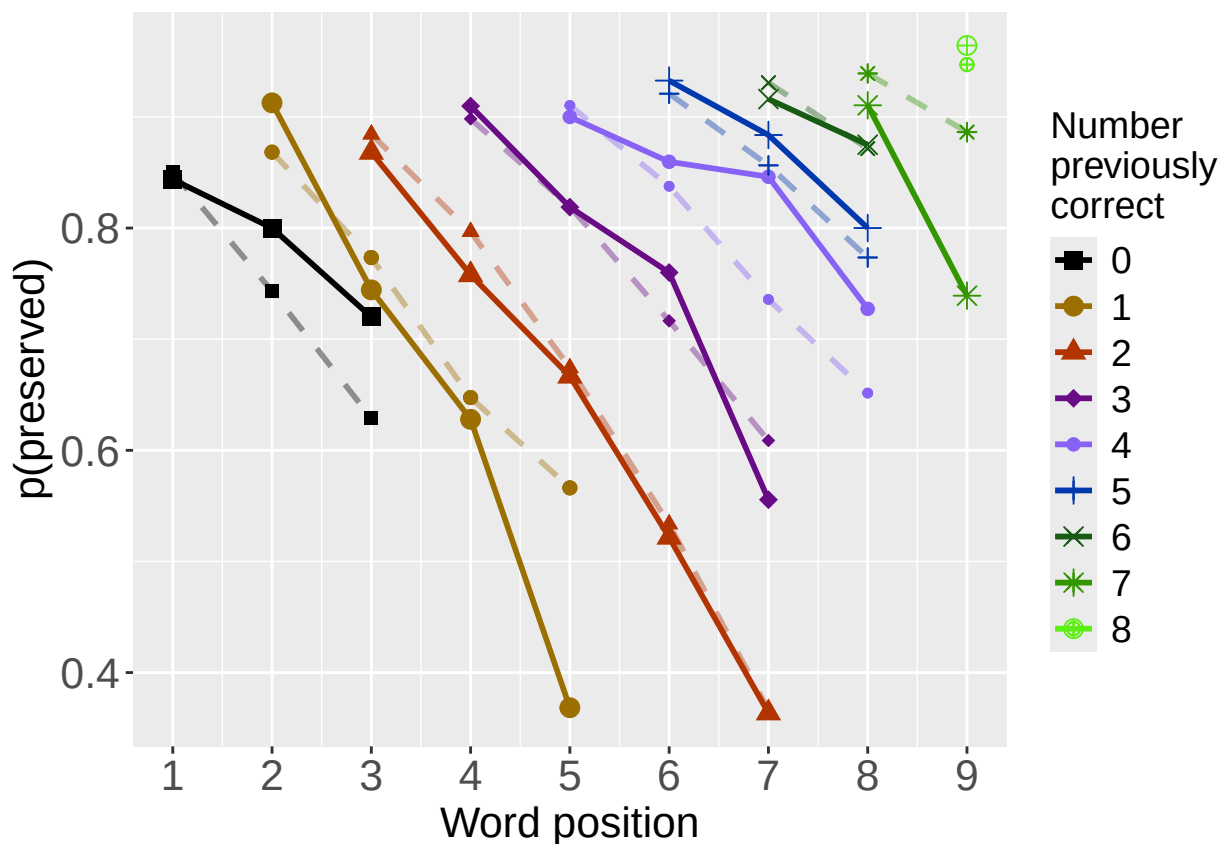
```
#####

write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                   RunLabel,"_",CurTask,
                                   "_best_model_single_term_deletions.csv"),
          row.names = TRUE)

# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

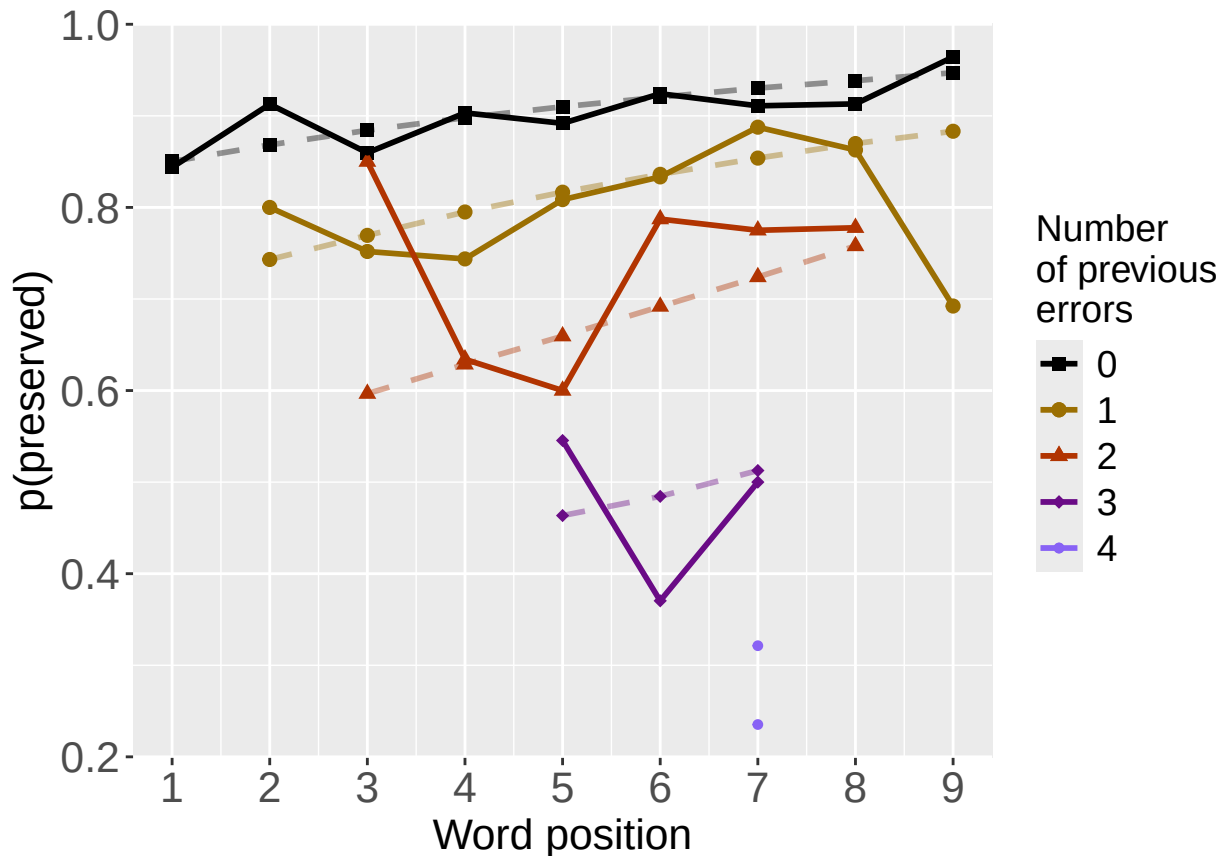
PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",
  RunLabel,"_",CurTask,
  "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
      family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.0539      -0.6622
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4073 Residual
## Null Deviance:      3523
## Residual Deviance: 3321 AIC: 3325
## log likelihood: -1660.591
## Nagelkerke R2: 0.08349509
## % pres/err predicted correctly: -1000.873
## % of predictable range [ (model-null)/(1-null) ]: 0.06430022

```



```

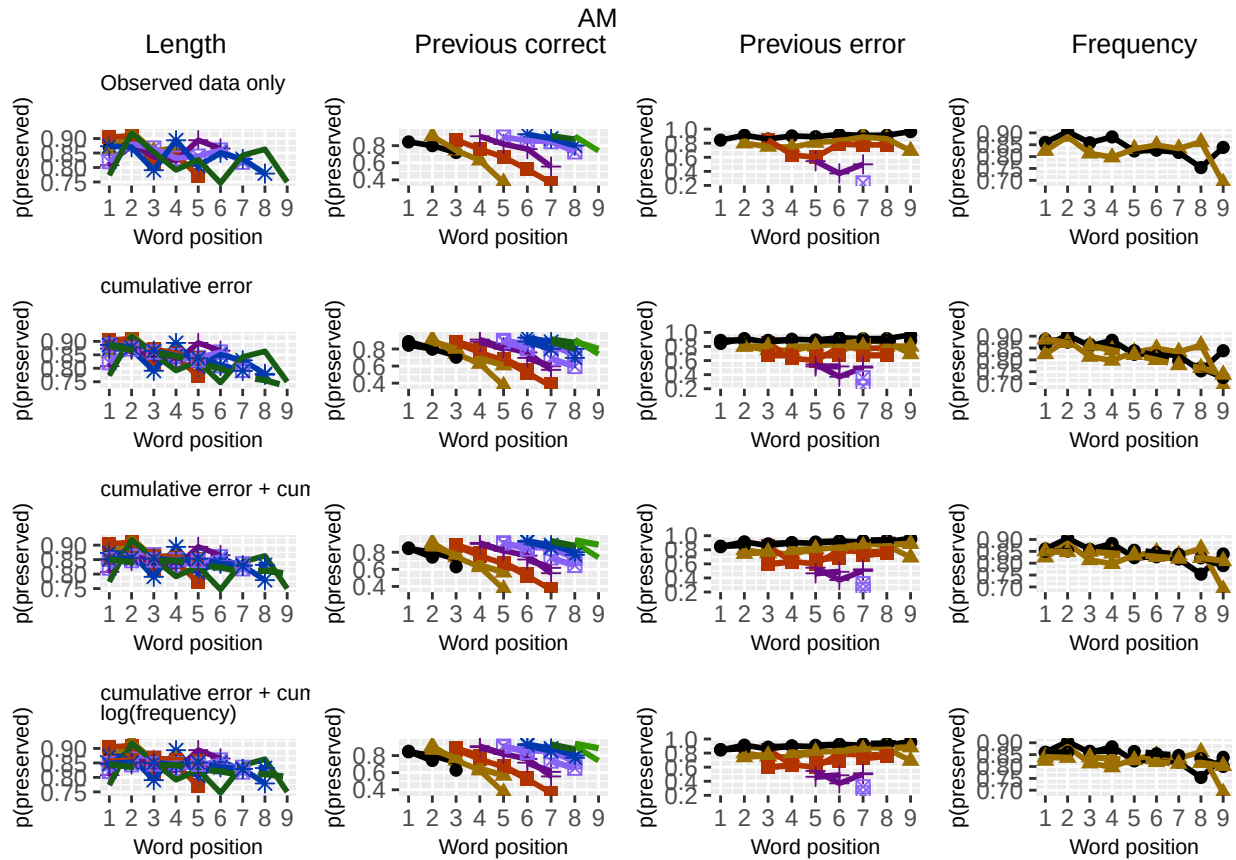
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      1.7407      -0.6738      0.1440
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4072 Residual
## Null Deviance:      3523
## Residual Deviance: 3287 AIC: 3293
## log likelihood: -1643.434
## Nagelkerke R2: 0.09728387
## % pres/err predicted correctly: -990.6031
## % of predictable range [ (model-null)/(1-null) ]: 0.07389215
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      log_freq
##      1.73043      -0.65924      0.14682      0.04764
##
## Degrees of Freedom: 4074 Total (i.e. Null); 4071 Residual
## Null Deviance:      3523
## Residual Deviance: 3283 AIC: 3291
## log likelihood: -1641.465
## Nagelkerke R2: 0.09885905
## % pres/err predicted correctly: -989.6327
## % of predictable range [ (model-null)/(1-null) ]: 0.07479847
## *****
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

```

```

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
ggsave(paste0(PlotName, ".tif"), plot=FactorPlot, width = 360, height=400, units="mm", device="tiff", compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot

```



```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                     CurTask,"_dominance_analysis_table.csv"),
          row.names = TRUE)
kable(DAContributionAverage)
```

	CumErr	CumPres	log_freq
McFadden	0.0561750	0.0094492	0.0025243
SquaredCorrelation	0.0471427	0.0079218	0.0021508
Nagelkerke	0.0471427	0.0079218	0.0021508
Estrella	0.0487986	0.0082104	0.0021875

	deviance	deviance_explained
CumErr + CumPres + log_freq	3282.929	240.0882
CumErr + CumPres	3286.868	236.1497
CumErr	3321.183	201.8345
null	3523.017	0.0000

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + CumPres + log_freq	6.814846	100.00000	1.640435
CumErr + CumPres	6.703053	98.35957	14.292770
CumErr	5.729023	84.06680	84.066795
null	0.000000	NA	0.000000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                model deviance deviance_explained percent_explained percent_of_explained_deviance
## CumErr + CumPres + log_freq CumErr + CumPres + log_freq 3282.929          240.0882          6.814846          100.00000
## CumErr + CumPres              CumErr + CumPres 3286.868          236.1497          6.703053          98.35956
## CumErr                        CumErr 3321.183          201.8345          5.729023          84.06680
## null                          null 3523.017          0.0000          0.000000          NA
##                                increment_in_explained
## CumErr + CumPres + log_freq          1.640435
## CumErr + CumPres                    14.292770
## CumErr                             84.066795
## null                               0.000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
```

```

NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents

```

```

##           Nagelkerke
## CumErr    0.82395268
## CumPres   0.13845526
## log_freq  0.03759206

```

```

sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)

```

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length

```

model	p_accounted_for	model_deviance
preserved ~ CumErr+CumPres	0.8684819	3286.868
preserved ~ CumErr+CumPres+log_freq	0.8701851	3282.929
preserved ~ CumErr	0.8789275	3321.183

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
sse_table <- sse_results_table(sse_results_list)
write.csv(sse_table, paste0(TablesDir, CurPat, "_", RunLabel, "_",
                           CurTask, "_sse_results_table.csv"), row.names = TRUE)

sse_table
```

```
##              model p_accounted_for model_deviance diff_CumErr+CumPres diff_CumErr+CumPres+log_freq diff_CumErr
## 1      preserved ~ CumErr+CumPres      0.8684819      3286.868              0.000000000              -0.001703225 -0.010445600
## 2 preserved ~ CumErr+CumPres+log_freq      0.8701851      3282.929              0.001703225              0.000000000 -0.008742375
## 3      preserved ~ CumErr      0.8789275      3321.183              0.010445600              0.008742375  0.000000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF, paste0(TablesDir, CurPat, "_", RunLabel, "_",
                                    CurTask, "_results_report_df.csv"), row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

model	diff_CumErr+CumPres	diff_CumErr+CumPres+log_freq	diff_CumErr
preserved ~ CumErr+CumPres	0.0000000	-0.0017032	-0.0104456
preserved ~ CumErr+CumPres+log_freq	0.0017032	0.0000000	-0.0087424
preserved ~ CumErr	0.0104456	0.0087424	0.0000000